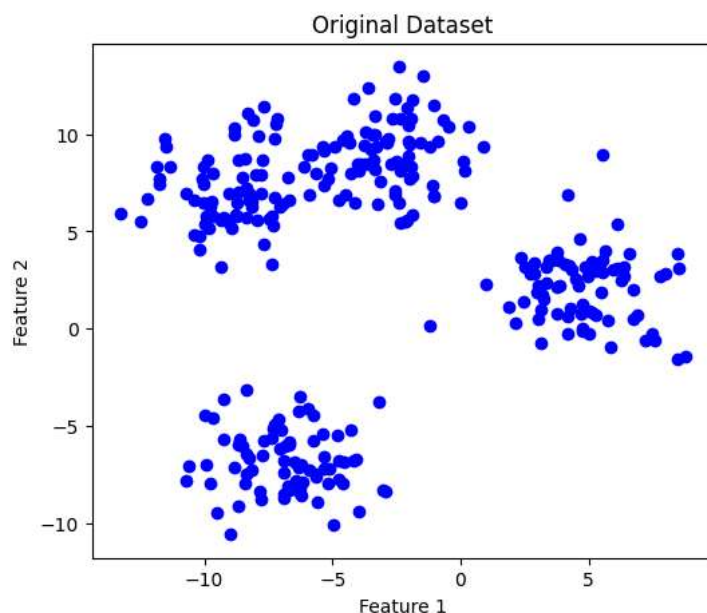


```
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

```
# Generate Dataset
X, y = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=1.8,
    random_state=42
)

plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c='blue')
plt.title("Original Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

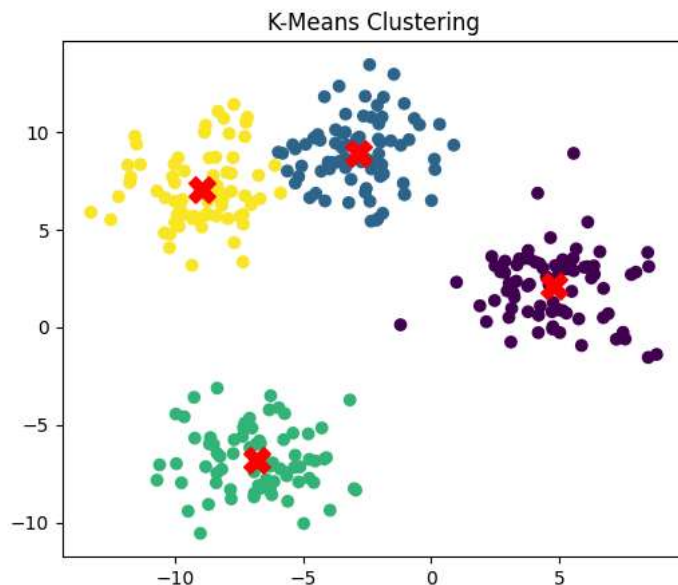


```
# Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=0)
labels = kmeans.fit_predict(X)

plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[0,0],
            kmeans.cluster_centers_[0,1],
            color='red',
            marker='x',
            s=200)

plt.title("K-Means Clustering")
plt.show()

print("Silhouette Score:", silhouette_score(X, labels))
```



Silhouette Score: 0.6353143639129922

```
# Optimization using Standardization + K-Means++
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans_opt = KMeans(
    n_clusters=4,
    init="k-means++",
    n_init=10,
    random_state=42
)

labels_opt = kmeans_opt.fit_predict(X_scaled)

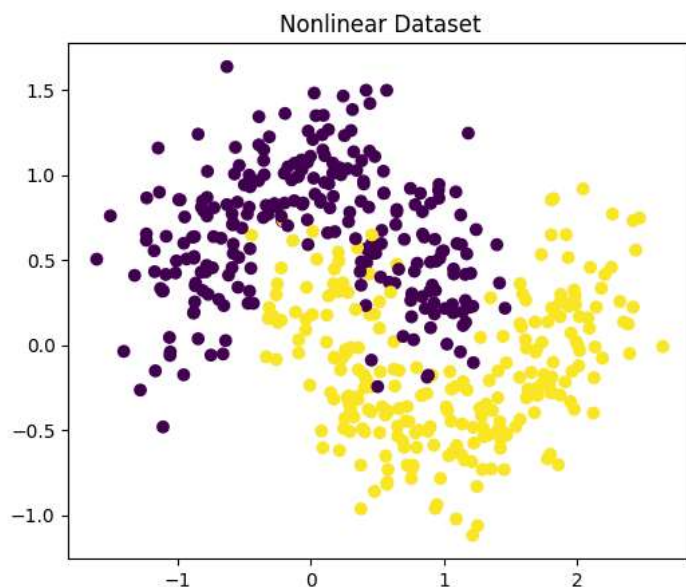
print("Optimized Silhouette Score:",
      silhouette_score(X_scaled, labels_opt))
```

Optimized Silhouette Score: 0.6420829927125842

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# Generate Nonlinear Dataset
X, y = make_moons(
    n_samples=500,
    noise=0.25,
    random_state=42
)

plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Nonlinear Dataset")
plt.show()
```



```
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Linear Kernel
linear = SVC(kernel='linear')

linear.fit(X_train, y_train)

pred = linear.predict(X_test)

print("Linear Kernel Accuracy:",
      accuracy_score(y_test, pred))
```

Linear Kernel Accuracy: 0.8533333333333334

```
# RBF Kernel (Optimized)
rbf = SVC(
    kernel='rbf',
    C=10,
    gamma=0.5
)

rbf.fit(X_train, y_train)

pred2 = rbf.predict(X_test)

print("RBF Kernel Accuracy:",
      accuracy_score(y_test, pred2))
```

RBF Kernel Accuracy: 0.96

```
import numpy as np

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import SimpleRNN
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
# Dummy Dataset
X = np.array([
    [1,2,3,4],
    [2,3,4,5],
    [4,5,6,7],
    [3,4,5,6]
])

y = np.array([0,1,1,0])

X = pad_sequences(X, maxlen=4)
```

```
# Simple RNN Model
model = Sequential([
    Embedding(input_dim=20, output_dim=8),
    SimpleRNN(16),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X, y, epochs=20, verbose=1)
```

```
Epoch 1/20
1/1 ————— 5s 5s/step - accuracy: 0.5000 - loss: 0.6829
Epoch 2/20
1/1 ————— 0s 74ms/step - accuracy: 0.5000 - loss: 0.6770
Epoch 3/20
1/1 ————— 0s 46ms/step - accuracy: 0.7500 - loss: 0.6711
Epoch 4/20
1/1 ————— 0s 44ms/step - accuracy: 0.7500 - loss: 0.6651
Epoch 5/20
1/1 ————— 0s 44ms/step - accuracy: 1.0000 - loss: 0.6591
Epoch 6/20
1/1 ————— 0s 49ms/step - accuracy: 1.0000 - loss: 0.6530
Epoch 7/20
1/1 ————— 0s 45ms/step - accuracy: 1.0000 - loss: 0.6467
Epoch 8/20
1/1 ————— 0s 44ms/step - accuracy: 1.0000 - loss: 0.6403
Epoch 9/20
1/1 ————— 0s 44ms/step - accuracy: 1.0000 - loss: 0.6338
Epoch 10/20
1/1 ————— 0s 47ms/step - accuracy: 1.0000 - loss: 0.6270
Epoch 11/20
1/1 ————— 0s 48ms/step - accuracy: 1.0000 - loss: 0.6199
Epoch 12/20
1/1 ————— 0s 46ms/step - accuracy: 1.0000 - loss: 0.6127
Epoch 13/20
1/1 ————— 0s 53ms/step - accuracy: 1.0000 - loss: 0.6052
Epoch 14/20
1/1 ————— 0s 45ms/step - accuracy: 1.0000 - loss: 0.5974
Epoch 15/20
1/1 ————— 0s 45ms/step - accuracy: 1.0000 - loss: 0.5894
Epoch 16/20
1/1 ————— 0s 43ms/step - accuracy: 1.0000 - loss: 0.5810
Epoch 17/20
1/1 ————— 0s 48ms/step - accuracy: 1.0000 - loss: 0.5723
Epoch 18/20
1/1 ————— 0s 45ms/step - accuracy: 1.0000 - loss: 0.5633
Epoch 19/20
1/1 ————— 0s 44ms/step - accuracy: 1.0000 - loss: 0.5539
Epoch 20/20
1/1 ————— 0s 44ms/step - accuracy: 1.0000 - loss: 0.5442
<keras.src.callbacks.history.History at 0x7b09bb535070>
```

```
# Improved LSTM Model
model2 = Sequential([
    Embedding(input_dim=20, output_dim=8),
    LSTM(16),
    Dense(1, activation='sigmoid')
])

model2.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
model2.fit(X, y, epochs=20, verbose=1)
```

```
Epoch 1/20
1/1 ————— 5s 5s/step - accuracy: 0.7500 - loss: 0.6924
Epoch 2/20
1/1 ————— 0s 45ms/step - accuracy: 0.7500 - loss: 0.6923
Epoch 3/20
1/1 ————— 0s 48ms/step - accuracy: 0.7500 - loss: 0.6921
Epoch 4/20
1/1 ————— 0s 58ms/step - accuracy: 0.7500 - loss: 0.6919
Epoch 5/20
1/1 ————— 0s 50ms/step - accuracy: 0.7500 - loss: 0.6918
Epoch 6/20
1/1 ————— 0s 51ms/step - accuracy: 0.7500 - loss: 0.6916
Epoch 7/20
1/1 ————— 0s 48ms/step - accuracy: 0.7500 - loss: 0.6914
Epoch 8/20
1/1 ————— 0s 47ms/step - accuracy: 0.7500 - loss: 0.6912
Epoch 9/20
1/1 ————— 0s 46ms/step - accuracy: 0.7500 - loss: 0.6910
Epoch 10/20
1/1 ————— 0s 46ms/step - accuracy: 0.7500 - loss: 0.6908
Epoch 11/20
1/1 ————— 0s 51ms/step - accuracy: 0.7500 - loss: 0.6906
Epoch 12/20
1/1 ————— 0s 46ms/step - accuracy: 0.7500 - loss: 0.6904
Epoch 13/20
1/1 ————— 0s 46ms/step - accuracy: 0.7500 - loss: 0.6902
Epoch 14/20
1/1 ————— 0s 47ms/step - accuracy: 0.7500 - loss: 0.6900
Epoch 15/20
1/1 ————— 0s 47ms/step - accuracy: 0.7500 - loss: 0.6898
Epoch 16/20
1/1 ————— 0s 48ms/step - accuracy: 0.7500 - loss: 0.6896
Epoch 17/20
1/1 ————— 0s 49ms/step - accuracy: 0.7500 - loss: 0.6893
Epoch 18/20
1/1 ————— 0s 47ms/step - accuracy: 0.7500 - loss: 0.6891
Epoch 19/20
1/1 ————— 0s 47ms/step - accuracy: 1.0000 - loss: 0.6888
Epoch 20/20
1/1 ————— 0s 46ms/step - accuracy: 1.0000 - loss: 0.6886
<keras.src.callbacks.history.History at 0x7b094ad25b80>
```

```
# Compare the models
```

```
acc_loss_acc_acc = model.evaluate(X, y, verbose=0)
```